# Executive Summary: Batch Experiment Runner & Catchment Network Calibration

**Project**: pyrrm (Python Rainfall-Runoff Models) **Date**: 19 February 2026

---

## Motivation

The pyrrm library currently supports calibrating individual rainfall-runoff models one at a time. Users who need to compare multiple models, objective functions, or algorithms across a catchment must write manual loops in notebooks, handle failures themselves, and aggregate results by hand. There is also no support for calibrating a regional network of subcatchments where upstream flows feed into downstream nodes.

This build addresses both gaps with two new capabilities:

1. **BatchExperimentRunner** -- automate combinatorial single-catchment experiments
2. **CatchmentNetworkRunner** -- calibrate a river network from headwaters to outlet

Both share a unified parallelization layer that supports sequential execution, multiprocessing, and Ray.

---

## What We Are Building

### Component 1: Unified Parallel Backend

A pluggable parallelization abstraction (`pyrrm/parallel.py`) used by both runners.

| Backend | Dependencies | Use Case |
|---|---|---|
| `SequentialBackend` | None | Debugging, small experiments |
| `MultiprocessingBackend` | stdlib only | Multi-core local machine |
| `RayBackend` | Ray (optional) | DAG-aware task scheduling, resource management |

Users select via a single parameter: `backend='sequential'`, `'multiprocessing'`, or `'ray'`.

The backend exposes two key methods:

- **`map()`** -- run a list of independent tasks in parallel (used by the batch runner)
- **`map_wavefronts()`** -- run tasks in dependency-ordered groups where each group runs in parallel but groups are sequential (used by the network runner)

### Component 2: BatchExperimentRunner

Automates combinatorial experiments for a single catchment. The user defines an `ExperimentGrid` with named axes:

- **Models**: e.g., GR4J, GR5J, GR6J, Sacramento
- **Objectives**: e.g., NSE, KGE, log-NSE, SDEB

- **Algorithms**: e.g., SCE-UA, DREAM, differential evolution
- **Flow transformations** (optional): e.g., sqrt, log, inverse

The runner generates all combinations, executes each calibration, and collects results into a `BatchResult` object with summary tables, comparison utilities, and save/load support.

**Key features**:

- Resume capability: skips already-completed experiments on restart
- Failure isolation: failed experiments are logged and skipped, not fatal
- Immediate persistence: each result is saved to disk as it completes
- Configurable via Python API or YAML file

**Example scale**: 4 models x 13 objectives x 2 algorithms = 104 calibrations, managed automatically.

### Component 3: CatchmentNetworkRunner

Calibrates rainfall-runoff models across a river network, respecting the upstream-to-downstream dependency structure. The network is defined as a directed acyclic graph (DAG) where nodes are subcatchments and edges are river links.

**Network definition** is provided via CSV or YAML files:

- A topology file defining subcatchments, their downstream connections, areas, and data file paths
- An optional link routing file defining Muskingum routing parameters (K, m, n_subreaches) and calibration bounds for each river reach

**Layered calibration configuration**: Each node's calibration is defined by five ingredients -- the RR model, objective function, flow transformation, optimization algorithm, and warmup period. These are configured in two layers:

1. **Network defaults** -- set once on the runner, apply to every node unless overridden
2. **Per-node overrides** -- set on individual nodes, override the default for that node only

| Ingredient | Network default example | Per-node override example |
|---|---|---|
| RR model | `GR4J` for all nodes | Junction uses `Sacramento` (more complex catchment) |
| Objective | `KGE(variant='2012')` | Headwater uses `NSE(transform='sqrt')` for low-flow focus |
| Flow transformation | `None` (no transform) | Specific node uses `FlowTransformation('log')` |
| Algorithm | `sceua_direct, max_evals=20000` | Complex node gets `max_evals=50000` or switches to `DREAM` |
| Warmup | 365 days | Sacramento node gets 730 days (needs longer spinup) |

This enables patterns ranging from fully uniform networks (set defaults, leave all overrides empty) to fully heterogeneous ones (different model/objective/algorithm at every node). The runner provides a `get_resolved_configs()` method that previews the effective configuration for each node after merging defaults and overrides, so the user can verify the setup before launching a long calibration.

**Three calibration strategies**:

| Strategy | Description | Routing Params | Speed |
|---|---|---|---|
| **Incremental** (default) | Calibrate each node sequentially from headwaters to outlet. At junctions, local RR model and incoming link routing parameters are calibrated jointly. | Calibrated at junctions | Moderate |
| **Residual** | Same sequential structure, but link routing parameters are fixed (user-provided). Only local RR models are calibrated. | Fixed | Fast |
| **Simultaneous** | All RR and routing parameters across the entire network are calibrated in a single optimization. | Calibrated globally | Slow but avoids error propagation |

**DAG-aware parallel scheduling**: The network topology is decomposed into wavefronts (dependency levels). All nodes in a wavefront are independent and can run in parallel. For example, in a network with 8 headwater catchments feeding into 4 junctions feeding into 2 sub-outlets feeding into 1 outlet:

```
Wavefront 0:  8 headwaters    (all in parallel)
Wavefront 1:  4 junctions     (all in parallel)
Wavefront 2:  2 sub-outlets   (all in parallel)
Wavefront 3:  1 outlet
```

**Link routing calibration**: Each river link between subcatchments has its own Nonlinear Muskingum router instance with three calibratable parameters:

- **K** -- storage constant (travel time through the reach, in days)
- **m** -- nonlinear exponent controlling the storage-discharge relationship
- **n_subreaches** -- number of numerical sub-reaches for routing stability

These are calibrated at the downstream junction node, because that is where observed flow data exists to constrain the routing. The parameter vector at a junction includes both the local RR model parameters and the routing parameters for all incoming links.

---

## Component 4: Network Data Preparation (NetworkDataLoader)

The existing `InputDataHandler` validates a single catchment's DataFrame. For network calibration, loading and aligning data across many catchments is a prerequisite that currently requires extensive manual work (in notebooks, each catchment requires 3+ `pd.read_csv()` calls, manual column renaming, sentinel value replacement, and date alignment via joins).

`NetworkDataLoader` (`pyrrm/network/data.py`) automates this for the entire network.

**Two supported data layouts**:

- **Single file per catchment**: one CSV with columns `date, rainfall, pet, observed_flow`
- **Separate files per variable**: individual CSVs for rainfall, PET, and flow (like the existing 410734 data with separate rain, PET, and flow files)

**Processing pipeline** (per catchment, then network-level checks):

| Step | Scope | What it does |
|---|---|---|
| File discovery | Per node | Resolve paths relative to `data_dir`; clear error if missing |
| Column detection | Per node | Auto-detect column names using `InputDataHandler`'s synonym lists |
| Sentinel replacement | Per node | Replace -9999, -999, etc. with NaN |
| Date parsing | Per node | Parse dates, set DatetimeIndex |
| Variable merge | Per node | If separate files, inner-join on date index |
| Column standardisation | Per node | Rename to `precipitation`, `pet`, `observed_flow` |
| Unit conversion | Per node | Convert flow to target units (ML/day, mm/day, cumecs) |
| Frequency validation | Network | Verify all catchments are daily frequency |
| Gap analysis | Per node | Missing value counts, longest gap, % complete |
| Missing value handling | Per node | Configurable: `warn` (default), `interpolate` (<=5 day gaps), `drop` |
| Junction overlap analysis | Network | Compute effective calibration period at each junction |

**No global date trimming**: Each catchment retains its full available record. Every node -- headwater or downstream -- uses its own maximum available period for calibration. The date overlap constraint only applies at junction aggregation time during calibration. At each junction, the runner computes the **intersection** of three periods: (a) the junction's own observed flow record, (b) the junction's input data period (precipitation, PET), and (c) each upstream node's simulation period. The effective calibration period at the junction is this intersection. Any of these can be the limiting factor depending on the data availability in the specific network.

For example:

- Headwater 1: inputs 1960-2020, observed 1965-2018 -- calibrates on 1965-2018 (53 years)
- Headwater 2: inputs 1990-2020, observed 1995-2020 -- calibrates on 1995-2020 (25 years)
- Junction: inputs 1950-2020, observed 1960-2020 -- effective period is 1995-2018 (limited by HW2 start and HW1 observed end)

The limiting factor can be any node in the network. The validation report shows exactly which record constrains each junction's effective calibration period.

**Validation report**: Before calibration, `NetworkDataLoader.validate()` produces a `DataValidationReport` with per-catchment data quality AND per-junction overlap analysis showing the effective calibration period at each junction. This ensures the user can see exactly how much data is available at every point in the network before committing to a potentially long calibration run.

```
loader = NetworkDataLoader('network_topology.csv', data_dir='./data/')
data = loader.load()
report = loader.validate()
print(report)   # per-catchment quality + junction overlap analysis
```

### Component 5: Network Visualization

All network objects (topology, configuration, data quality, calibration results) are visualisable in Jupyter notebooks through two complementary formats:

**Mermaid diagrams** -- generated as strings, rendered natively in Jupyter via `IPython.display.Markdown`. Three variants:

| Diagram | Stage | Shows |
|---|---|---|
| **Topology diagram** | Pre-calibration | Network DAG with node names, areas, model types, link routing params. Gauged nodes distinguished from ungauged. Optional wavefront grouping. |
| **Configuration diagram** | Pre-calibration | Same DAG, annotated with resolved calibration config (model, objective, algorithm) per node. Per-node overrides visually distinguished from defaults. |
| **Results diagram** | Post-calibration | Same DAG, nodes annotated with objective values, links annotated with calibrated K/m. Colour-coding by performance quality. |

**Styled DataFrames** -- pandas Styler objects that render as colour-coded HTML tables in Jupyter. Four variants:

| Table | Stage | Shows |
|---|---|---|
| **Config summary** | Pre-calibration | One row per node: model, objective, transformation, algorithm, warmup. Per-node overrides highlighted. Wavefront grouping via row colours. |

| Table | Stage | Shows |
|---|---|---|
| **Link config summary** | Pre-calibration | One row per link: routing method, initial params, bounds, whether calibrated. |
| **Data quality summary** | Pre-calibration | One row per node: input/observed periods, gap %, longest gap. Gap cells colour-coded green/amber/red. Junction effective calibration periods shown with limiting factor. |
| **Results summary** | Post-calibration | One row per node: best objective (colour-coded), runtime (bar sparkline), model, n_params, calibration period, status. Failed nodes in red. |

All visualisation methods are also available as plain DataFrames (without styling) for programmatic use.

---

## Architecture Diagram

```
                    +-----------------------+
                    |   ParallelBackend     |
                    |  (seq / mp / ray)     |
                    +-----------+-----------+
                                |
              +----------------+----------------+
              |                                 |
+----------v-----------+          +----------v-----------+
| BatchExperimentRunner|          | CatchmentNetworkRunner|
|                      |          |                       |
| - ExperimentGrid     |          | - CatchmentNetwork    |
|   (models x obj x   |          |   (DAG topology)      |
|    algo x transform) |          | - NetworkLink          |
| - YAML config        |          |   (per-link routing)  |
| - Resume / persist   |          | - 3 strategies        |
+----------+-----------+          | - Wavefront scheduling|
              |                   +----------+------------+
              |                              |
              |                   +----------v-----------+
              |                   | NetworkDataLoader    |
              |                   |                      |
              |                   | - Multi-file loading |
              |                   | - Sentinel cleanup   |
              |                   | - Temporal alignment |
              |                   | - Validation report  |
              |                   +----------+-----------+
              |                              |
              +----------------+----------------+
                                |
                    +-----------v-----------+
                    |   CalibrationRunner   |
                    |   (existing pyrm)     |
                    +-----------+-----------+
                                |
```

```
                    +----------------+----------------+
                    |                |                |
              SCE-UA           DREAM            SciPy
```

---

## New Files

| File | Purpose |
|---|---|
| `pyrrm/parallel.py` | Parallel backend abstraction (Sequential, Multiprocessing, Ray) |
| `pyrrm/calibration/batch.py` | ExperimentGrid, BatchExperimentRunner, BatchResult |
| `pyrrm/network/__init__.py` | Network module exports |
| `pyrrm/network/data.py` | NetworkDataLoader, CatchmentData, DataValidationReport |
| `pyrrm/network/topology.py` | CatchmentNode, NetworkLink, CatchmentNetwork (DAG with topological sort and wavefronts) |
| `pyrrm/network/runner.py` | CatchmentNetworkRunner, NetworkCalibrationResult |
| `pyrrm/network/visualization.py` | Mermaid diagram generators, styled DataFrame builders, network matplotlib plots |

**Modified files**: `pyrrm/__init__.py`, `pyrrm/calibration/__init__.py` (new exports only).

---

## Dependencies

| Dependency | Type | Purpose |
|---|---|---|
| numpy, pandas, scipy | Core (existing) | Numerical computation |
| PyYAML | New (core) | YAML config parsing |
| Ray | New (optional) | Parallel task scheduling |
| tqdm | New (optional) | Progress bars |

Ray is handled as an optional dependency using the same try/except pattern as SpotPy and PyDREAM. The library works fully without it; Ray only unlocks the `backend='ray'` option.

---

## Reuse of Existing Infrastructure

This build is designed to layer on top of the existing pyrrm codebase without modifying core components:

- **CalibrationRunner** -- both runners delegate every individual calibration to the existing runner, so all algorithms (SCE-UA, DREAM, PyDREAM, SciPy), checkpointing, and reporting work unchanged.
- **CalibrationResult / CalibrationReport** -- results from batch and network experiments use the same data structures, including save/load, summary tables, and report card visualisation.

- **NonlinearMuskingumRouter** -- link routing in the network runner reuses the existing Muskingum implementation (with Numba JIT acceleration). Each link gets its own router instance. The `set_parameters()` / `get_parameter_bounds()` interface already supports calibration integration.
- **InputDataHandler** -- the `NetworkDataLoader` reuses the existing column name detection logic (precipitation/rainfall/precip synonyms, PET/evapotranspiration synonyms, flow/discharge synonyms) and unit conversion functions (mm_to_ml_per_day, cumecs_to_ml_per_day) rather than reimplementing them.
- **BaseRainfallRunoffModel** -- all models (GR4J, GR5J, GR6J, Sacramento) work without modification.

---

## Implementation Phases

| Phase | Scope | Estimated Complexity |
|---|---|---|
| **Phase 0** | Unified parallel backend | Low -- thin abstraction over existing tools |
| **Phase 1a** | ExperimentGrid + BatchExperimentRunner | Medium -- core loop, failure handling, resume |
| **Phase 1b** | BatchResult + aggregation | Low -- dataclass with DataFrame export |
| **Phase 1c** | YAML config parser | Low -- mapping YAML keys to Python objects |
| **Phase 1d** | Wire up exports | Trivial |
| **Phase 2a** | NetworkDataLoader + validation | Medium -- multi-file loading, alignment, gap analysis, validation report |
| **Phase 2b** | CatchmentNetwork + topology | Medium -- DAG representation, topological sort, CSV/YAML parsing |
| **Phase 2c** | CatchmentNetworkRunner + strategies | High -- 3 strategies, junction objective wrappers, link routing calibration |
| **Phase 2d** | Flow aggregation with routing | Medium -- per-link router management, parameter dispatching |
| **Phase 2e** | NetworkCalibrationResult | Low -- dataclass with summaries |
| **Phase 2f** | Network visualization | Medium -- Mermaid generators, styled DataFrames, matplotlib plots |
| **Phase 3** | Demo notebook | Low -- usage examples |

---

## Example Usage (Preview)

### Batch experiment

```
from pyrrm.calibration.batch import ExperimentGrid, BatchExperimentRunner
from pyrrm.models import GR4J, GR5J, Sacramento
from pyrrm.calibration import NSE, KGE

grid = ExperimentGrid(
    models={'GR4J': GR4J(), 'GR5J': GR5J(), 'Sacramento': Sacramento()},
    objectives={'nse': NSE(), 'kge': KGE()},
    algorithms={'sceua': {'method': 'sceua_direct', 'max_evals': 20000}},
)

runner = BatchExperimentRunner(
    inputs=cal_inputs, observed=cal_observed,
    grid=grid, output_dir='./results/batch',
    backend='ray', max_workers=4,
)

batch_result = runner.run(resume=True)
print(batch_result.to_dataframe())
```

### Network calibration

```
from pyrrm.network import CatchmentNetwork, CatchmentNetworkRunner
from pyrrm.calibration import NSE

network = CatchmentNetwork.from_csv(
    topology_path='network_topology.csv',
    links_path='network_links.csv',
    data_dir='./data/',
)

runner = CatchmentNetworkRunner(
    network=network,
    default_model_class='GR4J',
    default_objective=NSE(),
    strategy='incremental',
    link_routing=True,
    backend='ray', max_workers=8,
    output_dir='./results/network',
)

result = runner.run(resume=True)
print(result.to_dataframe())
print(result.link_summary())
```